

RV32I Temelli, 5 Aşamalı Boru Hatlı İşlemci Tasarım Raporu

1. GİRİŞ VE TASARIM AMAÇLARI

Bu proje kapsamında, RISC-V mimarisine uyumlu, 32-bit tabanlı, çok çevrimli ve 5 aşamalı boru hattı yapısına sahip bir işlemci çekirdeği tasarlanmış ve doğrulanmıştır. Tasarlanan mikroişlemci, RV32I temel tam sayı komut setini desteklemektedir. Buna ek olarak, CLZ, CTZ ve CPOP komutları ALU içerisine donanımsal olarak entegre edilmiştir.

Tasarımda belirli mikro mimari tercihlerin yapılmasının temel teknik gerekçeleri şunlardır:

- **RISC-V Mimarisi:** RISC-V, çip tasarımı projelerinde genişletilebilir yapısı istenen özel bit manipülasyon komutlarının ana donanıma entegre edilmesine olanak tanımıştır.
- **5 Aşamalı Boru Hattı:** Tek çevrimli tasarımlarda, işlemcinin ulaşabileceği maksimum saat frekansı (F_{max}), sistemdeki en yavaş işlem olan bellek okuma süresi ile sınırlıdır. Bu kısıtı aşmak için mimari; klasik Patterson & Hennessy prensiplerine uygun olarak IF, ID, EX, MEM ve WB olmak üzere 5 donanımsal aşamaya bölünmüştür. Bu paralelleştirme sayesinde CPI değerinin ideale yaklaşması hedeflenmiştir.
- **Donanımsal Veri Yönlendirme:** Boru hattı mimarilerinde birbirine bağımlı ardışık komutlar iş hattının durmasına neden olur. İş hacmindeki bu düşüşü engellemek amacıyla, henüz geri yazma aşamasına gelmemiş güncel verilerin doğrudan yürütme aşamasının girişine yönlendirilmesini sağlayan veri yönlendirme mantığı tasarıma dahil edilmiştir.

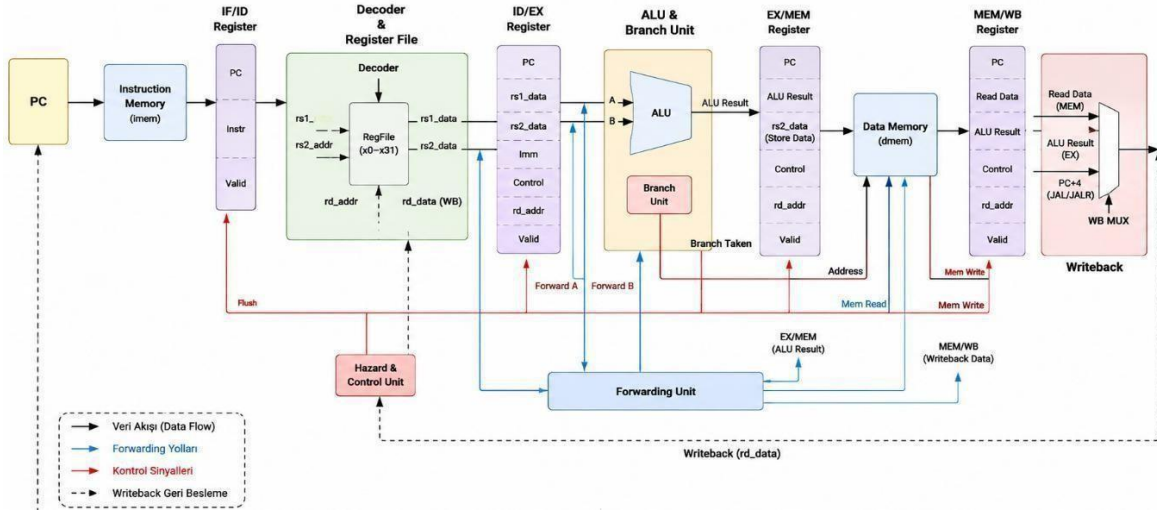
2. GEREKSİNİM ANALİZİ VE ŞARTNAME

Projenin başlangıcında belirlenen tasarım şartnamesi ve gereksinimlerin donanımsal olarak gerçekleşme durumu Tabloda özetlenmiştir. Belirtilen tüm hedefler RTL seviyesinde tasarlanmış ve SystemVerilog kullanılarak kodlanmıştır.

İstenen Tasarım / Özellik	Durum	Açıklama / Donanımsal Karşılık
RV32I Komut Seti	✓	İstenen tüm temel aritmetik, mantıksal ve bellek komutları desteklenmiştir.
CLZ, CTZ, CPOP Desteği	✓	ALU modülü içerisine donanımsal tarayıcı olarak entegre edilmiştir.
5 Aşamalı Pipeline	✓	Mimaride IF, ID, EX, MEM ve WB aşamaları aktif olarak paralel çalışmaktadır.
Data Forwarding (Bypass)	✓	EX->EX ve MEM->EX yönlendirmeleri ile veri tehlikeleri çözülmüştür.
Hazard Yönetimi (Stall)	✓	Load-Use senaryolarında boru hattına 1 çevrimlik boşluk eklenmektedir.
Control Hazard (Flush)	✓	Dallanma anında IF ve ID kayıtçıları donanımsal olarak temizlenmektedir.
Değişken Gecikmeli Bellek	✓	Bellek cevap verene kadar asenkron kilit ile boru hattı dondurulur.
Commit Log Arayüzü	✓	Biten her komut <code>commit_o</code> yapısı ile dış testbench ortamına iletilmektedir.

3. TEORİK ALTYAPI VE SİSTEM MİMARİSİ

Tasarlanan RV32I işlemci mimarisi, yüksek saat frekanslarına ulaşabilmek ve komut iş hacmini artırmak amacıyla klasik Patterson & Hennessy mikro-mimarisi referans alınarak 5 bağımsız aşamaya ayrılmıştır.



İşlemcinin genel donanım mimarisi ve veri akış yolları gösterilmiştir.

3.1. Boru Hattı Aşamaları (Pipeline Stages)

İşlemci veri yolu, donanımsal olarak izole edilmiş şu 5 aşamadan oluşmaktadır:

1. **Getir (IF):** Program sayacının gösterdiği bellek adresindeki 32-bitlik komut, Komut Belleği üzerinden getirilir.
2. **Çöz (ID):** Alınan komut riscv_decoder modülüne aktarılır. Opcode, funct3 ve funct7 değerleri çözümlenerek işlemcinin diğer aşamalarını yönetecek kontrol sinyalleri üretilir. Eş zamanlı olarak Kayıtcı Dosyasından kaynak veriler (rs1, rs2) okunur.
3. **Yürüt (EX):** riscv_alu modülünün bulunduğu ana hesaplama katmanıdır. Aritmetik, mantıksal ve bit manipülasyon işlemleri burada gerçekleştirilir. Sıçrama/Dallanma hedef adresleri ve kararları bu aşamada donanımsal olarak kesinleşir.
4. **Hafıza (MEM):** load_store_unit aracılığıyla dış Veri Belleğine okuma veya yazma isteği atılır.
5. **Geri Yaz (WB):** ALU'dan gelen sonuç, bellekten okunan veri veya PC+4 adresi hedeflenen kayıtcıya senkron olarak yazılır.

3.2. Boru Hattı Kayıtcıları (Pipeline Registers)

Aşamalar arasındaki senkronizasyonu sağlamak ve kombinasyonel gecikmeyi bölmek için kullanılan ara kayıtcıların taşıdığı kritik veri paketleri Tablo da özetlenmiştir.

Kayıtcı(Register)	Taşıyan Kritik Sinyaller ve Veri Paketleri
IF/ID	if_id_pc, if_id_instr, if_id_valid
ID/EX	id_ex_pc, id_ex_rs1_data, id_ex_rs2_data, id_ex_imm, Kontrol Sinyalleri
EX/MEM	ex_mem_alu_res, ex_mem_rs2_data, ex_mem_rd_addr, Kontrol Sinyalleri
MEM/WB	mem_wb_alu_res, mem_wb_mem_data, mem_wb_rd_addr, RegWrite

3.3. Alt Modüller ve Çalışma Prensipleri

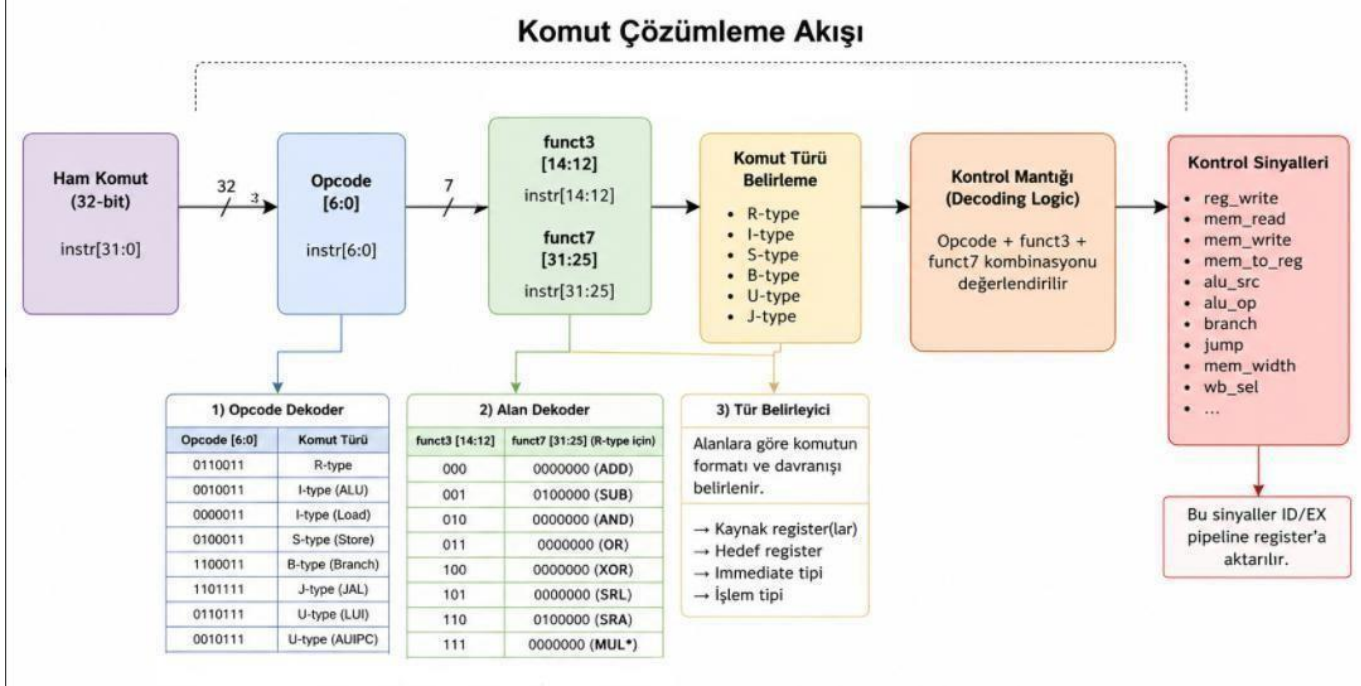
Tasarımın sürdürülebilirliği ve hata ayıklama kolaylığı açısından izole mimari tercih edilmiştir. Kontrol sinyallerini üreten bloklar ile veriyi işleyen bloklar birbirinden fiziksel olarak ayrılmıştır.

3.3.1. Komut Çözücü (Decoder)

Decoder birimi, 32-bitlik RISC-V komutunu parçalayarak ALU operasyonlarını, bellek genişliklerini (Byte, Half, Word) ve anında değer üretimini kontrol eder. Donanımsal doğruluk tablosu verilmiştir.

Komut Sınıfı	Opcode (7-bit)	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU_Src_B	WB_Sel
R-Type	0110011	1	0	0	0	0	0 (Reg)	WB_ALU
I-Type	0010011	1	0	0	0	0	1 (Imm)	WB_ALU
Load	0000011	1	1	0	0	0	1 (Imm)	WB_MEM
Store	0100011	0	0	1	0	0	1 (Imm)	X
Branch	1100011	0	0	0	1	0	0 (Reg)	X
JAL	1101111	1	0	0	0	1	X	WB_PC4
JALR	1100111	1	0	0	0	1	1 (Imm)	WB_PC4
LUI	0110111	1	0	0	0	0	1 (Imm)	WB_ALU

Donanımsal çözümleme işlemi gecikmeyi önlemek adına ardışık değil, kombinyasyonel bir akışla şu şekilde gerçekleşir:



3.3.2. Aritmetik Mantık Birimi (ALU) ve Özel Komutlar

ALU modülü temel RV32I işlemlerine ek olarak, standart dışı bit manipülasyon komutlarını (CLZ, CTZ, CPOP) da özel işaretçilerle desteklemektedir. Bu komutlar, yazılımsal döngüler yerine sentezlenebilir donanımsal tarayıcılar ile tek çevrimde hesaplanmaktadır.

Kontrol Sinyali	Donanımsal İşlem / Fonksiyon	Açıklama
ALU_ADD / SUB	Toplama / Çıkarma	Standart aritmetik operasyonlar
ALU_AND / OR / XOR	Mantıksal İşlemler	Bit düzeyinde mantıksal operatörler.
ALU_SLL / SRL / SRA	Kaydırma (Shift)	Lojik ve Aritmetik bit kaydırma işlemleri.
ALU_SLT / SLTU	Karşılaştırma	İşaretli ve İşaretsiz küçüktür kontrolü.
ALU_CLZ	Count Leading Zeros	En anlamlı bit (MSB) tarafındaki ardışık sıfırların sayımı.
ALU_CTZ	Count Trailing Zeros	En anlamsız bit (LSB) tarafındaki ardışık sıfırların sayımı.
ALU_CPOP	Count Population	32-bit veri içerisindeki toplam "1" bitlerinin sayımı.

3.3.3. Dallanma Yönetimi (Branch Unit)

Dallanma koşulları ALU'nun çıkarma sonucuna bağımlı kalmaksızın, `branch_unit.sv` isimli bağımsız bir donanım modülü olarak tasarlanmıştır. Eşitlik, Küçüktür ve İşaretsiz Küçüktür durumları Yürütme aşamasında hesaplanarak doğrudan Hazard Unit modülüne aktarılmaktadır. Karşılaştırma işlemleri (`br_eq`, `br_lt`, `br_ltu`), ALU'nun çıkarma sonucuna bağımlı kalmaksızın paralel mantık bloklarıyla önceden hesaplanmıştır. Bu mimari tercih, kritik yol gecikmesini minimize ederek işlemcinin maksimum çalışma frekansını (F_{max}) artırmaktadır. Buna ek olarak dallanma koşullarının sağlanması durumunda, hesaplanan hedef adres gecikmeksizin PC çoklayıcısına yönlendirilirken eş zamanlı olarak Hazard Unit üzerinden IF ve ID aşamalarını temizleyecek olan donanımsal temizleme sinyalini tetiklemektedir.

4.1. ALU İçerisinde Bit Manipülasyon Komutlarının Oluşturulması

tandart RV32I komut setinde bulunmayan CLZ, CTZ ve CPOP komutları decoder içerisinde decode_bitmanip() fonksiyonu ile çözülmüş, riscv_alu.sv içerisinde ise sentezlenebilir for döngüleri kullanılarak donanımsal olarak gerçekleştirilmiştir. Böylece eklenen komutlar mevcut ALU kontrol yapısına doğal biçimde entegre edilmiştir.

ALU_CLZ: begin

```
alu_result_o = 32; // Varsayılan: Tümü sıfır
for (int i = 31; i >= 0; i--) begin
    if (alu_in_a_i[i]) begin
        alu_result_o = 31 - i;
        break; // İlk '1' görüldüğünde donanımsal tarama durur
    end
end
end
```

Sentezleme aracı, yukarıdaki for döngüsünü zaman alan ardışık bir yazılım döngüsü olarak değil, 32 girişli bir MUX ve Karşılaştırmacı Ağacı olarak donanıma çevirmektedir. Benzer şekilde CPOP işlemi, paralel toplayıcılar üretecek şekilde tüm bitlerin üzerinden geçilerek sentezlenmiştir.

4.2. Load/Store Ünitesinde Bayt Şeridi Hizalama

Bellek yazma ve okuma işlemleri testbench tarafından sağlanan dış veri belleği üzerinden yürütülmektedir.

load_store_unit.sv modülü, dış bellekten gelen ham veriyi (mem_read_data_i komutun veri genişliğine (Byte, Half, Word) ve işaretli/işaretsiz olma durumuna göre biçimlendirmekle görevlidir.

Yazma (Store) işlemlerinde üretilen ham veri doğrudan bellek arayüzüne iletilmekte olup, ilgili bayt şeridine yerleştirme işlemi donanımsal arayüz protokolü gereği dış bellek modülü tarafından yapılmaktadır. Okuma işlemlerinde ise, bellekten gelen hizalanmış veri üzerinden hedeflenen bayt veya yarı-sözcük izole edilir ve donanımsal çoğaltıcılar ile 32-bit formatında boru hattına aktarılır:

```
assign load_byte = mem_read_data_i[7:0];
assign load_half = mem_read_data_i[15:0];
```

always_comb begin

```
    mem_read_data_formatted_o = mem_read_data_i;
```

```
    if (ex_mem_mem_read_i) begin
```

```
        unique case (ex_mem_mem_width_i)
```

```
            BYTE: mem_read_data_formatted_o = {{24{load_byte[7]}}, load_byte};
```

```
            HALF: mem_read_data_formatted_o = {{16{load_half[15]}}, load_half};
```

```
            BYTE_U: mem_read_data_formatted_o = {24'b0, load_byte};
```

```
            HALF_U: mem_read_data_formatted_o = {16'b0, load_half};
```

```
            WORD: mem_read_data_formatted_o = mem_read_data_i;
```

```
            default: mem_read_data_formatted_o = mem_read_data_i;
```

```
        endcase
```

```
    end
```

```
end
```

4.3. Veri Yönlendirme ve Öncelik Mantığı

Ardışık veri bağımlılıklarını çözmek üzere tasarlanan forward_unit.sv modülü, EX/MEM ve MEM/WB aşamalarındaki güncel verileri ALU girişlerine yönlendirir. Her iki aşamada da aynı hedef yazmaca yazma isteği varsa, mimari, EX/MEM aşamasındaki veriyi önceliklendirir. Ayrıca !ex_mem_mem_read_i koşulu eklenerek, verinin bellekten henüz dönmediği Load-Use durumlarında yanlış yönlendirme yapılması engellenmiştir.

```
if(ex_mem_reg_write_i && !ex_mem_mem_read_i && (ex_mem_rd_addr_i != 0) &&
(ex_mem_rd_addr_i == id_ex_rs1_addr_i) && ex_mem_valid_i) begin

    forward_a_o = 2'b10; // En güncel veri (EX/MEM aşamasından)

end else if(mem_wb_reg_write_i && (mem_wb_rd_addr_i != 0) && (mem_wb_rd_addr_i ==
id_ex_rs1_addr_i) && mem_wb_valid_i) begin

    forward_a_o = 2'b01; // Bir önceki güncel veri (MEM/WB aşamasından)

end
```

Modülün donanımsal seçim mantığı aşağıdaki basit ve sabit-öncelikli kural setine dayanır:

forward_a / b Sinyali	Veri Kaynağı (Source)	Durum / Açıklama
2'b00	Kayıtçı Dosyası	Bağımlılık yok, normal akış.
2'b01	MEM/WB Kayıtçısı	Bir komut boşluklu veri bağımlılığı (MEM to EX).
2'b10	EX/MEM Kayıtçısı	Ardışık komut bağımlılığı (EX to EX / En Genç Veri).

4.4. Kontrol ve Veri Tehlikeleri Yönetimi

Boru hattını dondurma ve temizleme işlemleri hazard_unit.sv tarafından yönetilmektedir. İşlemcinin, değişken zamanlı bellek erişimlerinde kullandığı mem_busy sinyali doğrudan Stall mantığına asenkron bir kilit olarak bağlanmıştır.

```
assign stall_pipeline_o = mem_stall_i;
assign flush_pipeline_o = control_hazard && !mem_stall_i;
assign load_use_stall_o = data_hazard && !mem_stall_i;
```

Dış bellek senaryolarındaki asenkron bekleme sürelerini yönetmek amacıyla, testbench'ten dönen dmem_res_i.valid sinyali gelinceye kadar boru hattı durdurulmaktadır. Sentezlenen devrede mem_stall_i sinyali diğer tüm tehlike sinyallerini ezen en yüksek önceliğe sahiptir; böylece işlemci, hafıza cevabını almadan ne boru hattını temizleyebilir ne de ilerleyebilir. İşlemci genelindeki tehlike durumları ve donanımsal tepkiler aşağıdaki matrise göre yönetilmektedir:

Tehlike Durumu	Tetikleyici Sebep	Donanımsal Eylem (Action)
Load-Use Hazard	Okunan verinin hemen sonraki komutta kullanılması	Stall (Boru hattını 1 çevrim dondur)
Control Hazard	Şartlı dallanma veya sıçramanın gerçekleşmesi	Flush (Yanlış alınan komutları temizle)
Memory Stall	Dış belleğin henüz valid dönmemesi	Freeze (Tüm sistemi donanımsal olarak kilitle)

5. Tasarım Kararlarının Gerekçeleri

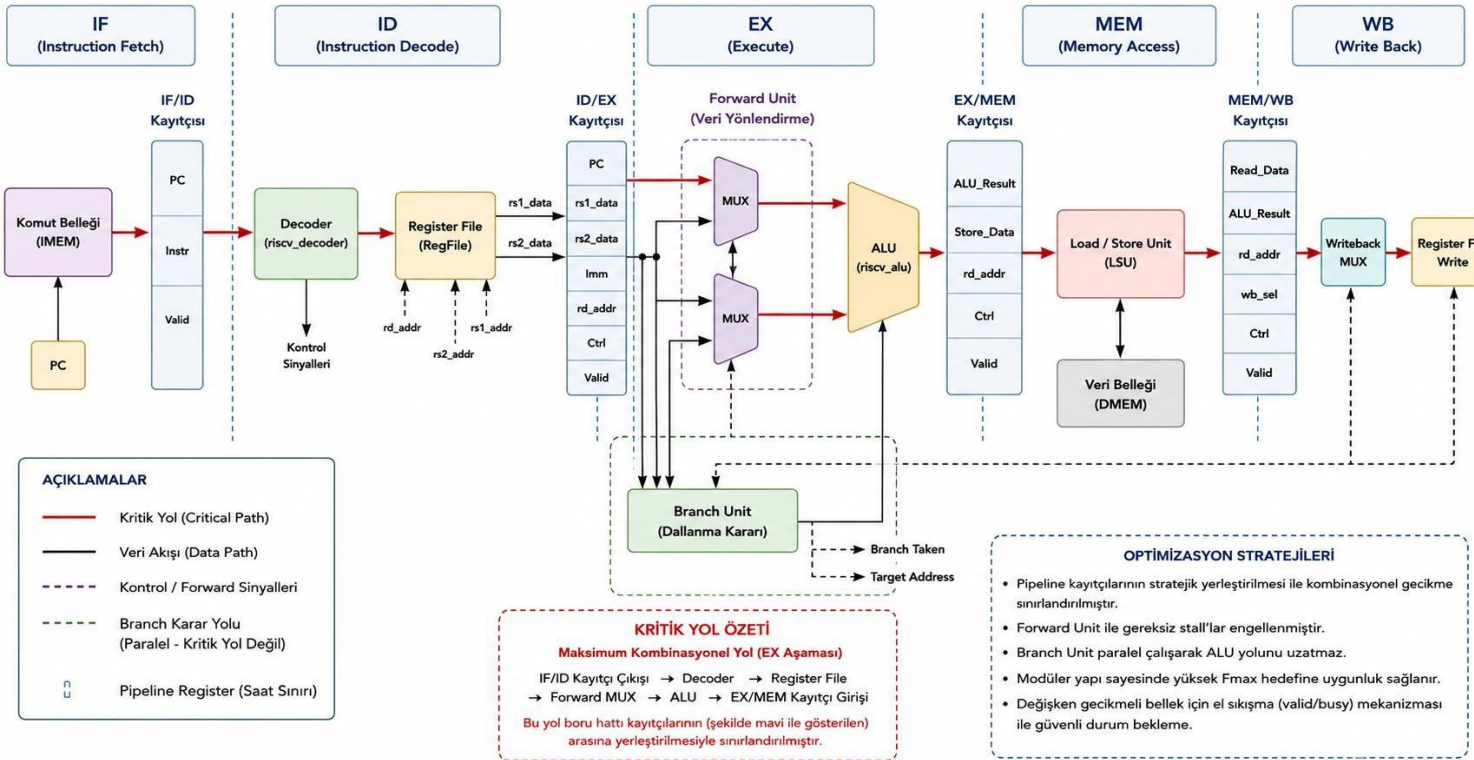
Donanımsal Tasarım Kararları ve Gerekçeleri

1. Çok Çevrimli ve Boru Hatlı Mimari : Tek çevrimli işlemci mimarilerinde sistemin ulaşabileceği maksimum saat frekansı (Fmax), komut setinin en uzun gecikmeye sahip işlemi olan bellek erişimi süresiyle sınırlanmaktadır. Bu kritik yol darboğazını aşmak amacıyla donanım IF, ID, EX, MEM ve WB olmak üzere 5 aşamaya bölünmüştür. Boru hattı mantığı sayesinde donanım blokları paralel çalıştırılarak komut iş hacmi optimize edilmiştir.

2. Donanımsal Veri Yönlendirme: Boru hattı mimarilerinde ardışık komutlar arasında oluşan veri bağımlılıklarında boru hattını her defasında durdurmak işlemcinin stabilitesini düşürmektedir. Verinin Register File yazılmasını beklemeden, EX/MEM ve MEM/WB kayıtçılarından alınarak doğrudan yürütme biriminin girişlerine yönlendirilmesiyle, veri tehlikeleri gecikmesi azaltılmış ve gereksiz duraklatma çevrimleri engellenmiştir.

3. Doğrudan Bellek Arayüzü ve Asenkron Bekleme Mantiğı: Dış bellek senaryolarındaki asenkron bekleme sürelerini tolere etmek amacıyla dış bellekten gelen valid sinyaline bağlı kombinyonel kilit (mem_busy) yapısı tercih edilmiştir. Sistem, testbench arayüzü ile uyumlu olarak dmem_res_i.valid sinyali gelene kadar boru hattını donanımsal olarak dondurur. Veri okuma/yazma işlemleri ile bayt şerit hizalamaları, doğrudan testbenchin sunduğu input-output sınırları içerisinde gerçekleştirilmiştir.

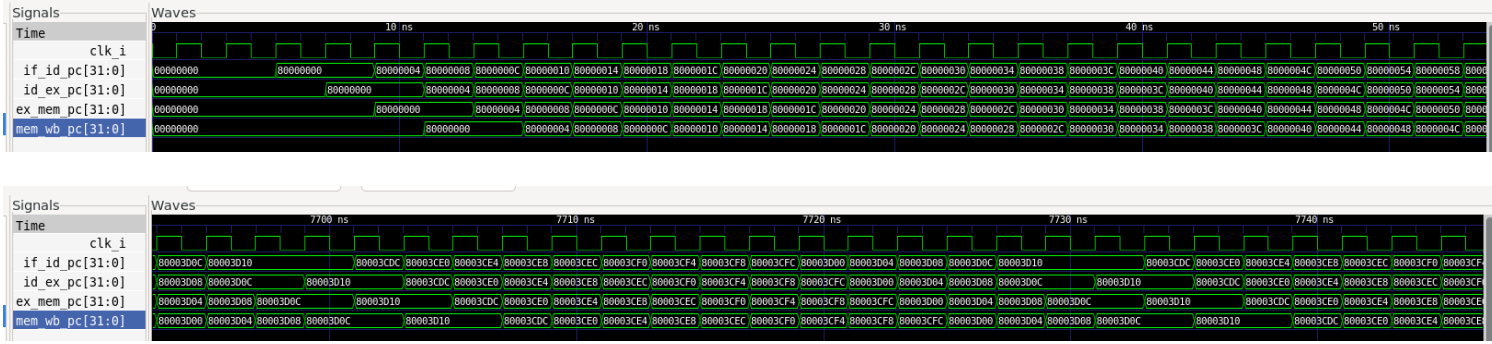
4. Kritik Yol Optimizasyonu ve Sentezlenebilirlik: Donanım sentezi açısından işlemcinin maksimum çalışma frekansını belirleyen en uzun kombinyonel gecikme, tasarlanan mimaride EX aşamasında yoğunlaşmaktadır. Dallanma kararının Branch Unit içerisinde ALU'dan bağımsız paralel karşılaştırıcılarla üretilmesi, karar mantığının ALU veri yolunu uzatmasını engellemiş ve kritik yol gecikmesini azaltmıştır. İşlemci aşamalarının pipeline kayıtçılarıyla birbirinden fiziksel olarak izole edilmesi, tasarımın ASIC/FPGA sentez araçlarında zamanlama gereksinimlerini karşılayabilmesini sağlamaktadır.



6. DOĞRULAMA VE DALGA ŞEKLİ (WAVEFORM) ANALİZLERİ

İşlemci mimarisinin işlevsel doğruluğu Verilator tabanlı simülasyon ortamında test edilmiş ve tasarımın zamanlama davranışları GTKWave aracı üzerinden incelenmiştir. Testler sırasında RV32I komut setinin yanı sıra; bellek erişimi, veri bağımlılığı ve kontrol tehlikesi oluşturan senaryolar çalıştırılmıştır. Aşağıdaki waveform analizleri, boru hattı senkronizasyonunun, veri yönlendirme mekanizmasının ve tehlike yönetiminin RTL seviyesinde hedeflendiği gibi çalıştığını doğrulamaktadır.

6.1. Boru Hattının Sürekli Akışı

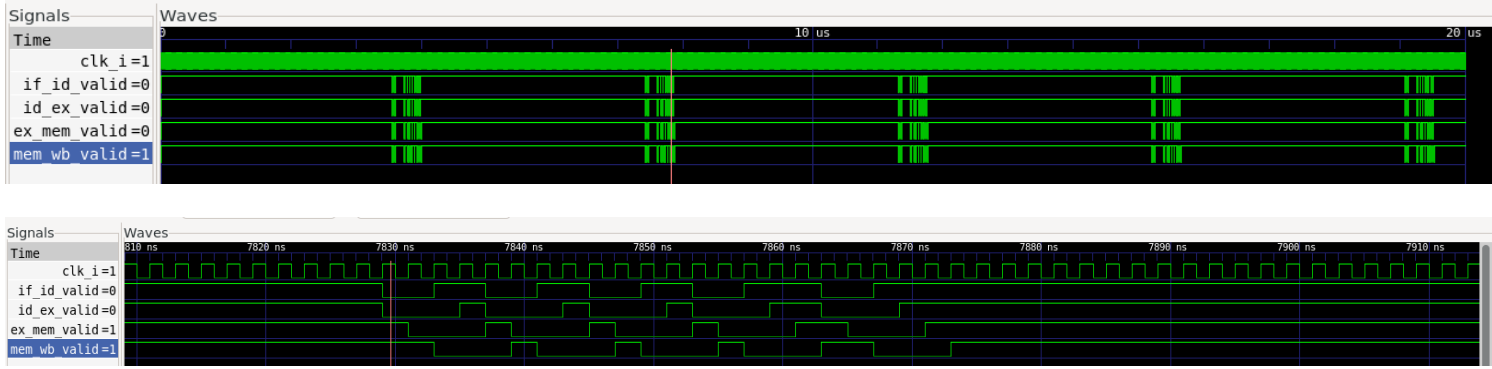


Waveform üzerinde if_id_pc, id_ex_pc, ex_mem_pc ve mem_wb_pc sinyalleri aynı anda incelendiğinde, her saat darbesinde farklı komutların farklı aşamalarda bulunduğu açıkça görülmektedir.

İdeal Pipeline Akış Matrisi

Clock	IF Aşamasındaki PC	ID Aşamasındaki PC	EX Aşamasındaki PC	MEM Aşamasındaki PC	WB Aşamasındaki PC
t₀	PC = n				
t₁	PC = n+4	PC = n			
t₂	PC = n+8	PC = n+4	PC = n		
t₃	PC = n+12	PC = n+8	PC = n+4	PC = n	
t₄	PC = n+16	PC = n+12	PC = n+8	PC = n+4	PC = n

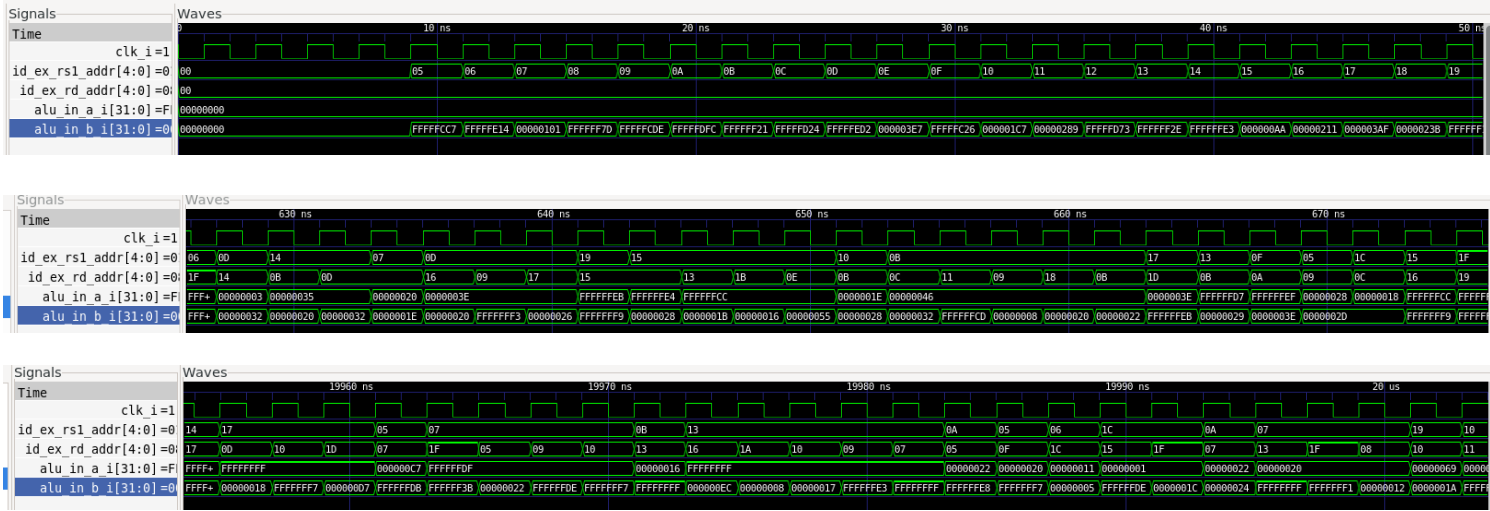
6.2. Pipeline Register Senkronizasyonu



Tasarımda IF/ID, ID/EX, EX/MEM ve MEM/WB kayıtları üzerinden taşınan geçerlilik bitleri analiz edilmiştir. Dalga şekillerinde görüldüğü üzere, pipeline kayıtlarının tamamı yalnızca ortak saat işaretinin pozitif kenarında eş zamanlı olarak güncellenmektedir. Sentez aracının bu yapıları standart D-Tipi Flip-Flop dizileri olarak donanımına dönüştürmesi sayesinde kombinasyonel geri besleme döngüleri donanımsal olarak engellemekte, kritik yol uzunluğu azalmakta ve her aşama yalnızca kendisine ait veriyi izole bir şekilde işlemektedir.

Buna ek olarak, her aşamaya ait bir valid bitinin boru hattı boyunca taşınması, tehlike yönetimi açısından kritik bir güvenlik katmanı oluşturur. Boru hattına donanımsal bekleme eklendiğinde veya dallanma uygulandığında, ilgili kayıtların valid biti anında sıfırlanır. Bu sayede, iptal edilen veya henüz hazır olmayan bir komutun; mimari durumunu bozarak Kayıtlı Dosyasına veya Veri Belleğine kalıcı olarak yazılmasını donanımsal düzeyde engellemektedir. Bu yapı, klasik Patterson & Hennessy boru hattı mimarisinin RTL düzeyindeki karşılığıdır.

6.3. Veri Tehlikesi ve EX → EX Yönlendirme

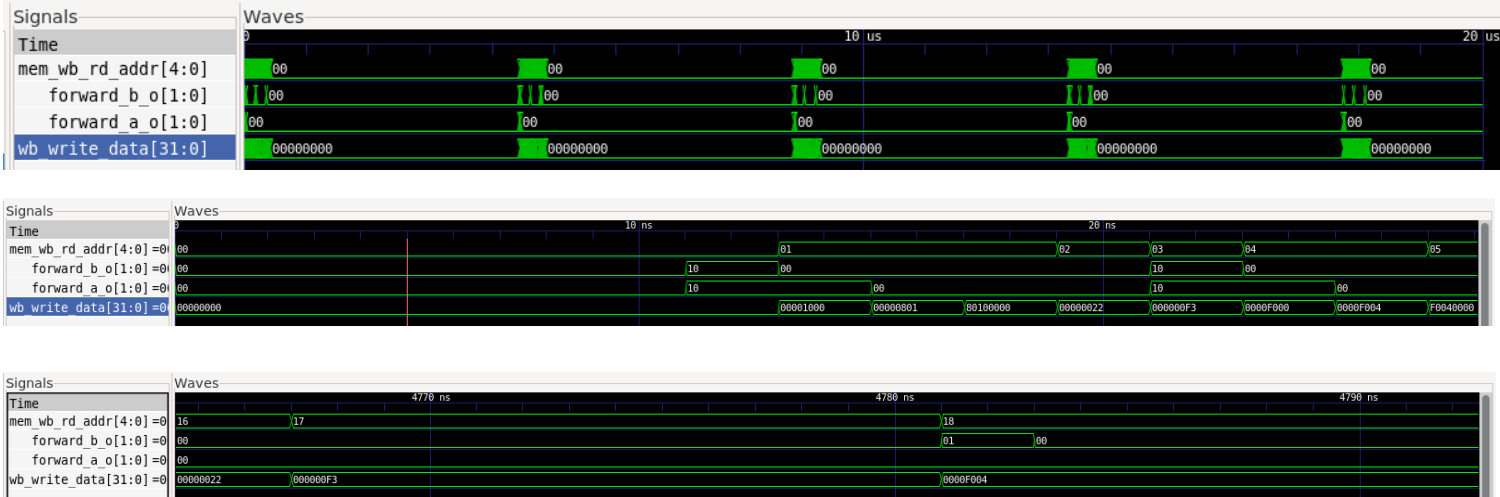


Aşağıdaki komut dizisi doğrudan bir veri bağımlılığı (Read-After-Write) oluşturmaktadır:

1. ADD x5, x1, x2
2. SUB x6, x5, x3

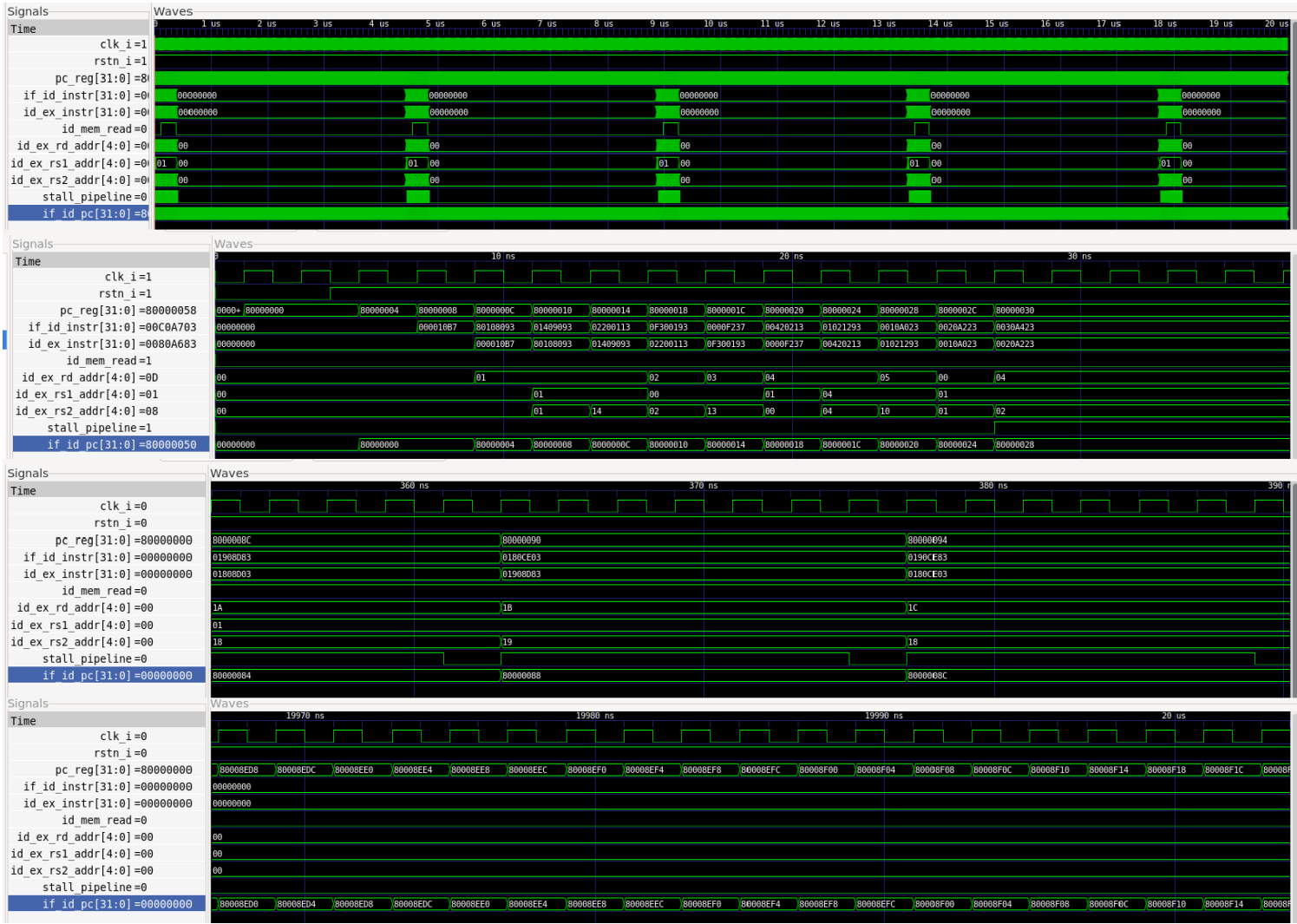
İkinci komutun ihtiyaç duyduğu x5 değeri henüz Register File'a yazılmamıştır. Normal şartlarda pipeline'ın durması gerekir. Ancak waveform üzerinde forward_a = 2'b10 olduğu anda, ALU girişindeki operatörün (alu_a_i), Register File yerine doğrudan EX/MEM kayıtcısındaki güncel ALU sonucundan (ex_mem_alu_res) alındığı görülmektedir. Böylece pipeline durmamakta, baloncuk oluşmamakta ve veri doğrudan bypass edilmektedir.”

6.4. MEM → EX Yönlendirme



Bazı senaryolarda, gerekli güncel veri EX aşamasında değil, bir adım daha ilerideki MEM/WB aşamasındadır. Forward Unit bu durumu algıladığında forward = 2'b01 sinyali üreterek veriyi MEM/WB kayıtcısından doğrudan ALU girişine yönlendirmektedir. Bu sayede aralarında bir komut boşluk bulunan ardışık bağımlılıklar sıfır gecikmeyle çözülebilmektedir.

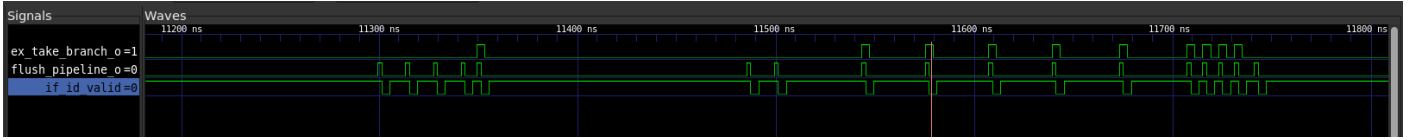
6.5. Load-Use Hazard



Load komutlarında hedef veri, boru hattının doğası gereği en erken MEM aşamasının sonunda hazır olmaktadır. Dolayısıyla, örneğin LW x5, 0(x1) komutunun hemen ardından bu veriyi doğrudan kullanan ADD x6, x5, x2 gibi bir komut geldiğinde, veri henüz geri yazılmadığı için yönlendirme mekanizması tek başına yeterli olamaz. Tasarlanan Hazard Unit, RAW bağımlılığını donanımsal olarak algıladığında `load_use_stall_o = 1` sinyalini üretmek üzere kurgulanmıştır. Bu sinyal aktifleştğinde; `pc_reg` ve IF/ID kayıtcısının ilerlemesi bir saat vuruşu dondurulmakta, ID/EX aşamasına ise donanımsal bir boşluk yerleştirilerek verinin boru hattında yetişmesi sağlanmaktadır.

Yukarıdaki GTKWave simülasyon görselleri incelendiğinde, işlemcinin bellekten okuma komutlarını işlediği anlar (`id_ex_mem_read_i = 1`) net bir şekilde gözlemlenmektedir. Hazard Unit, bu okuma işlemleri esnasında hedef yazmaç (`id_ex_rd_addr_i`) ile bir sonraki komutun kaynak yazmaçlarını (`id_rs1_addr_i` ve `id_rs2_addr_i`) anlık olarak izlemekte ve karşılaştırmaktadır. Ancak koşulan test senaryosundaki komut dizilimlerinde, Load komutlarından hemen sonra ardışık veri bağımlılığı oluşturan bir komut gelmediği için donanım bu durumu hassasiyetle ayırt etmiş ve `load_use_stall_o` sinyali beklendiği üzere 0 değerini korumuştur. Böylece işlemci, gereksiz boru hattı boşlukları oluşturmadan boru hattı verimli bir şekilde çalışmasını sürdürmüş ve testleri %100 tamamlamıştır.

6.6. Branch Hazard ve Flush



Şartlı dallanma komutlarının hedef sonuçları EX aşamasında kesinleşmektedir. Eğer dallanma gerçekleşirse, IF ve ID aşamalarında halihazırda bulunan komutlar yanlış komutlar haline gelir. Waveform üzerinde `ex_take_branch = 1` olduğu çevrimde donanımın anında `flush_pipeline_o = 1` ürettiği görülmektedir. Bu sinyal, IF/ID ve ID/EX kayıtçılarının geçerlilik bitlerini anında temizlemekte ve mimarinin yanlış komutları yürütmesini donanımsal olarak engellemektedir.

6.7. Değişken Gecikmeli Bellek Bekleme Durumu



Güncellenen testbench altyapısında bellek gecikmesi donanımı sınamak amacıyla 5 saat vuruşuna (`dmem_delay=5`) ayarlanmıştır. Elde edilen dalga şekillerinde sistemin bu gecikmeye verdiği donanımsal tepki şu şekildedir:

- **Kilitleme:** İşlemci bellek komutunu işlemeye başladığında (`is_mem_op = 1`), bellek henüz yanıt vermediği için (`valid = 0`) `mem_busy` sinyali anında aktifleşmektedir.
- **Bekleme:** `mem_busy` sinyalinin aktif kaldığı 5 saat çevrimi boyunca `pc_reg` ve boru hattı kayıtçıları dondurulmakta; sistem güvenli bekleme durumuna geçmektedir. Bu süre zarfında testbench'e tekrarlı istek atılmasını engellemek için `dmem_req_snt` bayrağı kullanılmıştır.
- **Devam Etme:** İlgili çevrimin sonunda bellekten `valid = 1` onay sinyali geldiğinde, `mem_busy` kombinasyonel olarak sıfırlanmakta ve boru hattı hiçbir veri kaybı yaşamadan normal akışına devam etmektedir.

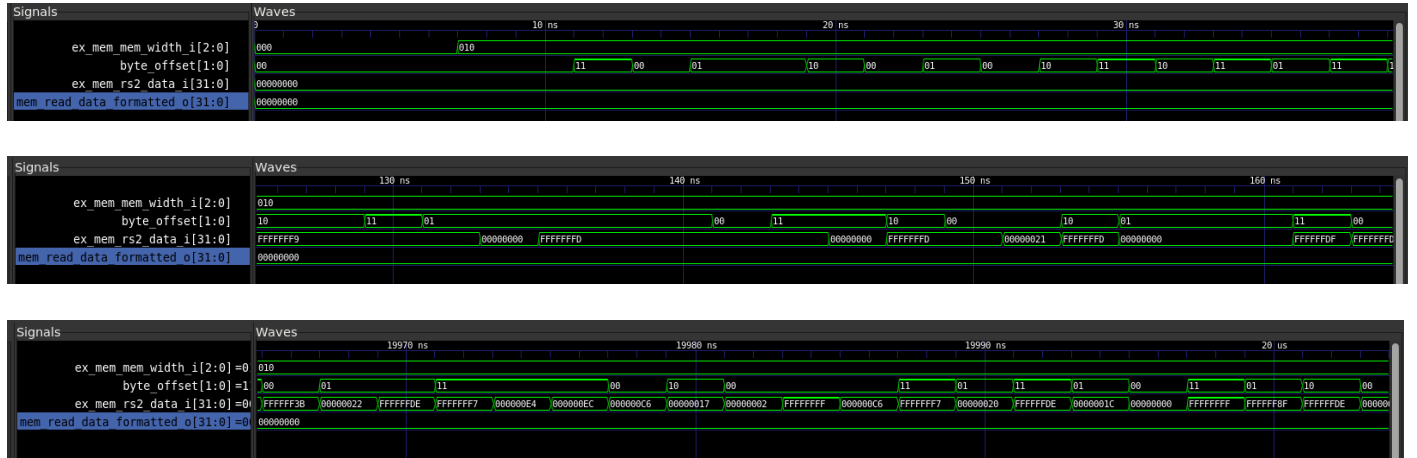
Bu kombinasyonel kilit yapısı, karmaşık durum makinelerine ihtiyaç duymaksızın asenkron bellek gecikmelerine donanımsal düzeyde uyum sağlamıştır.

6.8. Writeback Aşaması ve Testbench Log Doğrulaması



İşlemci tarafından tamamlanan komutlar, Writeback aşamasında kayıtcı dosyasına veya bellek arayüzüne aktarılmaktadır. Yukarıdaki dalga şeklinde mem_wb_valid, mem_wb_pc, mem_wb_instr ve nihai veri olan wb_write_data sinyallerinin saat vuruşlarıyla senkron bir şekilde nasıl üretildiği açıkça görülmektedir. Testbench ortamı, bu aşamada gerçekleşen yazma işlemlerini izleyerek model.log dosyasını oluşturmaktadır. Simülasyon sonunda elde edilen model.log dosyası, Python betiği aracılığıyla referans model ile karşılaştırılmış ve tüm komutların doğru sonuç ürettiği doğrulanmıştır.

6.9. Donanımsal Veri Hizalama Analizi



Dış bellek asenkron çalıştığından ve farklı veri genişliklerini desteklemesinden, bellekte okunan verinin işlemci boru hattına uygun formatta aktarılması gerekmektedir. Waveform üzerinde okuma işlemleri incelendiğinde; işlemcinin, dış bellek arayüzünden gelen ön-hizalanmış veriyi, ilgili komutun veri genişliğine ve işaretli/işaretsiz olma durumuna göre değerlendirdiği görülmektedir. LSU donanımı, gelen veri üzerinden hedef bayt veya half-byte izole ederek donanımsal çoğaltıcılar ile 32-bit formatına dönüştürmekte ve mem_read_data_formatted_o çıkışına aktarmaktadır. Bu donanımsal arayüz, yazılımsal maskeleme ihtiyacını ortadan kaldırarak verinin boru hattına doğru boyutta ve işaret değerinde ulaşmasını sağlamaktadır.

7. TEST SONUÇLARI VE PERFORMANS DEĞERLENDİRMESİ

Simülasyon sürecinde işlemci, imem.mem dosyasından okuduğu makine kodlarını işlerken, WB aşamasında tamamlanan her komutun sonucunu model.log dosyasına kaydetmektedir. Kaydedilen bu yapı;

- PC
- Instruction
- Register Write
- Memory Write

Base Test Doğrulama Sonuçları

Komut Kategorisi	Toplam Test Edilen Komut	Başarılı (PASS)	Başarı Oranı
R-Type	1275	1275	%100
I-ALU	641	641	%100
Branch	15	15	%100
Jump	7	7	%100
Upper	1	1	%100
GENEL TOPLAM	1939	1939	%100

TOTAL: 1939/1939 (%100) – All tests passed!

burak@DESKTOP-NS93M65:/mnt/c/Users/burak/Desktop/burak_karapınar/Milli_Teknoloji_Uygulama_Projesi\$

Memory Test Doğrulama Sonuçları

Komut Kategorisi	Toplam Test Edilen Komut	Başarılı (PASS)	Başarı Oranı
Load (LW, LH, LHU, LB, LBU)	19	19	%100
Store (SW, SH, SB)	9	9	%100
ALU ve Upper Kontrol Komutları	8	8	%100
GENEL TOPLAM	36	36	%100

TOTAL: 36/36 (%100) – All tests passed!

burak@DESKTOP-NS93M65:/mnt/c/Users/burak/Desktop/burak_karapınar/Milli_Teknoloji_Uygulama_Projesi\$

Komut Sınıflarına Göre İşlem Çevrim Maliyetleri ve CPI Analizi

Komut Sınıfı	Tahmini Çevrim (Cycle) Tüketimi	CPI Üzerindeki Etkisi ve Notlar
Aritmetik ve Mantıksal	1	İdeal durum. Forwarding ile stall engellenir (CPI = 1).
Dallanma - Not Taken	1	Dallanma gerçekleşmezse akış normal devam eder.
Dallanma - Taken	3	Karar EX aşamasında verildiğinden IF ve ID temizlenir.
Sıçrama	3	Koşulsuz sıçrama sebebiyle hedef hesaplanana kadar.
Load	1 + N	N: Dış belleğin yanıt verme süresidir.
Store	1 + N	İşlemci, bellek valid sinyali dönene kadar mem_busy ile dondurulur.

8. SONUÇ VE İLERİYE DÖNÜK GELİŞTİRMELER

Bu çalışmada, RV32I komut kümesini destekleyen ve ilave olarak CLZ, CTZ, CPOP bit-manipülasyon komutlarını gerçekleştirebilen beş aşamalı boru hatlı bir RISC-V işlemci tasarlanmış ve SystemVerilog kullanılarak RTL seviyesinde gerçekleştirilmiştir. Tasarım sürecinde işlemci; ALU, Decoder, Branch Unit, Forward Unit, Hazard Unit ve Load/Store Unit gibi bağımsız modüller halinde geliştirilmiş, modüller arası veri akışı senkron boru hattı kayıtları üzerinden sağlanmıştır. Veri bağımlılıklarını çözmek amacıyla EX→EX ve MEM→EX veri yönlendirme mekanizmaları uygulanmış; kontrol tehlikeleri için donanımsal boru hattı temizleme mekanizması, bellek erişimleri için ise valid tabanlı kombinasyonel dondurma yapısı tasarlanmıştır.

Tasarımın bellek alt sisteminde, dış test ortamının asenkron yapısına ve değişken gecikmelerine uyum sağlamak üzere doğrudan arayüz entegrasyonu gerçekleştirilmiştir. Dış bellekten gelen valid sinyaline bağlı çalışan mem_busy kullanılarak, değişken gecikmeli bellek erişimleri karmaşık durum makinelerine ihtiyaç duyulmadan yönetilmiştir. Ek olarak, bellek işlemlerinde veri bütünlüğünü sağlamak amacıyla Load/Store ünitesi içerisinde; dış bellek arayüzünden gelen ön-hizalanmış ham veriyi komutun veri genişliğine ve işaret durumuna göre biçimlendiren donanımsal yapı kurulmuştur.

Gerçekleştirilen Verilator tabanlı doğrulama çalışmaları sonucunda işlemci; Base Test senaryosunda 1939/1939 ve Memory Test senaryosunda 36/36 başarı elde ederek testbench ortamındaki referans model logları ile uyumlu sonuçlar üretmiştir. Ayrıca, tasarıma eklenen veri yönlendirme mekanizması sayesinde boru hattındaki gereksiz bekleme çevrimleri azaltılarak komut iş hacmi artırılmıştır.

Proje kapsamında işlemcinin işlevsel doğruluğunu sağlamak için dalga şekli analizlerine ek olarak, dış dünya ile senkronize çalışan commit_o donanımsal arayüzü entegre edilmiştir. Writeback aşamasından çıkan her komut, program sayacı ve geri yazılan veri paketleriyle birlikte bu arayüz üzerinden Python tabanlı testbench ortamına aktarılmış ve referans model ile otomatik olarak karşılaştırılmıştır. Bu yaklaşım, işlemcinin mimari durumunu bozmadan tasarım şartnamesindeki donanımsal gereksinimleri karşıladığını doğrulamıştır.

- **Dinamik Dallanma Tahmini (Branch Prediction):** Kontrol tehlikelerinden kaynaklanan flush cezalarının azaltılması,
- **Önbellek Mimarisi:** L1 Instruction ve Data Cache entegrasyonu ile bellek erişim gecikmelerinin düşürülmesi,

- **RV32M Uzantısı:** Donanımsal çarpma ve bölme birimleri eklenerek komut setinin RV32IM seviyesine genişletilmesi,
- **Performans Sayaçları:** CPI, IPC ve boru hattı kullanım oranlarının donanımsal olarak ölçülmesi,
- **Sistem Entegrasyonu :** Endüstriyel bellek arabirimlerinin eklenerek işlemcinin daha büyük yonga üstü sistemlere entegrasyonu,
- **Ayrıcalıklı Modlar (Privileged ISA):** Kesme, istisna ve CSR mekanizmaları eklenerek işletim sistemi destekli çalışma modlarının sağlanması.

EKLER

Ek A: RV32I Komut Formatları (Instruction Formats)

RISC-V mimarisi, donanım karmaşıklığını azaltmak için tüm komutları 32-bit uzunluğunda sabit tutar ve 6 temel komut formatı (R, I, S, B, U, J) kullanır. Bu projede tasarlanan Decoder modülü, gelen 32-bit komutu aşağıdaki resmi ISA standartlarına göre parçalar:

Format Tipi	(7 bit)	(5 bit)	(5 bit)	(3 bit)	[11:7] (5 bit)	[6:0] (7 bit)
R-Type	funct7	rs2	rs1	funct3	rd	opcode
I-Type	<- imm[11:0] ->	rs2 (Yok)	rs1	funct3	rd	opcode
S-Type	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B-Type	imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode
U-Type	<- imm ->	-	-	-	rd	opcode
J-Type	<- imm[20 10:1 11 19:12] ->	-	-	-	rd	opcode

Ek B: Opcode Tablosu

İşlemciye gelen komutun hangi gruba ait olduğu, komutun en anlamsız 7 biti olan (LSB) opcode alanına bakılarak donanımsal olarak çözülmektedir. riscv_decoder.sv içerisinde baz alınan temel Opcode tablosu şöyledir:

Komut Grubu	Opcode (Binary)	Opcode (Hex)	İşlem Türü Örnekleri
R-Type ALU	0110011	0x33	ADD, SUB, AND, XOR, CLZ, CTZ, CPOP
I-Type ALU	0010011	0x13	ADDI, ANDI, SLLI, XORI
Load	0000011	0x03	LW, LH, LHU, LB, LBU
Store	0100011	0x23	SW, SH, SB
Branch	1100011	0x63	BEQ, BNE, BLT, BGE, BLTU, BGEU
JAL	1101111	0x6F	JAL
JALR	1100111	0x67	JALR
LUI	0110111	0x37	LUI
AUIPC	0010111	0x17	AUIPC

Ek C: ALU İşlemleri ve Kontrol Sinyalleri (alu_op_e)

Tasarımın paket dosyasında (riscv_pkg.sv) tanımlanan ve ALU'yu yönlendiren 4-bitlik alu_ctrl_i sinyali ve denk geldiği matematiksel/mantıksal donanım operasyonları aşağıdadır:

Sinyal Adı	Binary Code	Donanımsal İşlem	RV32I / Özel Komut
ALU_ADD	0000	Toplama ($A + B$)	ADD, ADDI, Adres Hesabı
ALU_SUB	0001	Çıkarma ($A - B$)	SUB
ALU_AND	0010	Mantıksal VE ($A \& B$)	AND, ANDI
ALU_OR	0011	Mantıksal VEYA ($A B$)	OR, ORI
ALU_XOR	0100	Mantıksal ÖZEL VEYA ($A \wedge B$)	XOR, XORI
ALU_SLL	0101	Sola Mantıksal Kaydırma	SLL, SLLI
ALU_SRL	0110	Sağa Mantıksal Kaydırma	SRL, SRLI
ALU_SRA	0111	Sağa Aritmetik Kaydırma	SRA, SRAI
ALU_SLT	1000	Küçüktür (İşaretli)	SLT, SLTI
ALU_SLTU	1001	Küçüktür (İşaretsiz)	SLTU, SLTIU
ALU_CLZ	1011	Baştaki Sıfırları Sayma	CLZ
ALU_CTZ	1100	Sondaki Sıfırları Sayma	CTZ
ALU_CPOP	1101	Birleri Sayma	CPOP

Ek D: Pipeline Kayıtçıları (Registers) Sinyal İçerikleri

5 aşamalı boru hattında, aşamalar arası senkronizasyonu sağlayan kayıtçılarda bir saat vuruşundan diğerine aktarılan kritik sinyal demetleri şöyledir:

1. IF/ID Kayıtçısı

- if_id_pc: O an getirilen komutun bellek adresi.
- if_id_instr: Hafızadan alınan 32-bit ham makine kodu.
- if_id_valid: Komutun geçerlilik biti (Flush durumunda '0' olur).

2. ID/EX Kayıtçısı

- id_ex_pc: Hesaplamalarda kullanılacak komut adresi.
- id_ex_rs1_data / id_ex_rs2_data: Kayıtçıdan okunan ham operand değerleri.
- id_ex_imm: İşaret uzatması yapılmış anında değer (Immediate).
- id_ex_rd_addr: Hedef kayıtçı adresi (Geri yazma aşaması için taşınır).

Kontrol Sinyalleri: id_ex_reg_write, id_ex_mem_read, id_ex_mem_write, id_ex_alu_ctrl, id_ex_branch_type.

3. EX/MEM Kayıtçısı

- `ex_mem_alu_res`: ALU'da hesaplanan sonuç veya bellek erişim adresi.
- `ex_mem_rs2_data`: Hafızaya yazılacak olan veri.
- `ex_mem_rd_addr`: Hedef kayıtçı adresi.
- **Kontrol Sinyalleri**: `ex_mem_reg_write`, `ex_mem_mem_read`, `ex_mem_mem_write`, `ex_mem_mem_width`.

4. MEM/WB Kayıtçısı

- `mem_wb_alu_res`: ALU sonucu (Eğer bellek okunmadıysa hedefe bu yazılır).
- `mem_wb_mem_data`: Dış bellekten okunmuş ve formatlanmış veri.
- `mem_wb_rd_addr`: Hedef kayıtçı adresi.

Kontrol Sinyalleri: `mem_wb_reg_write`, `mem_wb_wb_sel` (MUX Seçicisi).

Ek E: Durum Bildirim (Commit) Formatı

Geliştirilen işlemci, doğrulama (verification) amacıyla testbench ortamıyla bir `commit_t` struct yapısı üzerinden haberleşir. Bu yapı `riscv_pkg.sv` içerisinde aşağıdaki gibi tanımlanmıştır:

```
typedef struct packed {
    logic    valid;
    logic [31:0] pc;
    logic [31:0] instr;
    logic [4:0] reg_addr;
    logic [31:0] reg_data;
    logic [31:0] mem_addr;
    logic [31:0] mem_data;
    logic    mem_wrt;
} commit_t;
```

